

Dictionary Search Project

Final Report

Ertan Tunç Türk Arda Eryılmaz Ömer Akçakaya

Yiğit Ali Ergül Yiğit Karakozak

December 2025

1 Introduction

1.1 Problem Statement

Our dictionary has 41,307 words. If we look at each word one by one, it takes a long time. We need faster ways to find words.

Our question is: How can we find a word quickly in a big dictionary? We want to test different methods and see which one is the fastest.

1.2 Project Objectives

What we want to do:

- Make a dictionary app with buttons and text boxes
- Write our own data structures (HashTable, ArrayList, LinkedList, Entry)
- Make three search methods: Linear, Binary, and Hash Search
- Measure how fast each method is
- Add a feature that suggests words when user makes mistakes

2 Algorithms and Data Structures

2.1 Search Algorithms

2.1.1 Our Linear Search

Linear Search is the simplest method. It looks at each word one by one from start to end.

Why we use compareTo(): We use compareTo() to compare words. This method works the same way in all our search classes.

Listing 1: Our Linear Search Code

```
1 public int search(T[] array, T target) {
2     for (int i = 0; i < array.length; i++) {
3         if (array[i].compareTo(target) == 0) return i;
4     }
5     return -1;
6 }
```

Why we made it: Linear Search is slow, but we need it to compare with other methods. It shows us how much faster the other methods are.

2.1.2 Our Binary Search

Binary Search is faster than Linear Search. But it needs the words to be in order (A to Z). So we also made a sort method.

How it works: It looks at the middle word. If the word we want comes before, it looks in the first half. If it comes after, it looks in the second half. It keeps doing this until it finds the word.

Safe math: We use $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ because it is safer with big numbers.

Listing 2: Our Binary Search Code

```
1 public int search(T[] sortedArray, T target) {
2     int left = 0, right = sortedArray.length - 1;
3     while (left <= right) {
4         int mid = left + (right - left) / 2;
5         int cmp = sortedArray[mid].compareTo(target);
6         if (cmp == 0) return mid;
7         else if (cmp < 0) left = mid + 1;
8         else right = mid - 1;
9     }
10    return -1;
11 }
```

Note: We sort the words once when the app starts. After that, every search is fast.

2.1.3 Our Hash Search

Hash Search is different. It does not look at words one by one. It uses a special number (hash code) to go directly to the word.

How it works: We give it a word. It makes a number from the word. This number tells us where to look. Very fast!

Listing 3: Our Hash Search Code

```
1 public class HashSearch<T extends Comparable<T>> implements Search<T> {
2     private HashTable<T, String> hashTable;
3
4     public HashSearch(HashTable<T, String> hashTable) {
5         this.hashTable = hashTable;
6     }
7
8     public int search(T[] array, T target) { // since we need to follow
9         interface we have to keep array param here
10        String result = hashTable.get(target);
11        return result != null ? 1 : -1;
12    }
```

Note: We do not use the array here. The HashTable already has all the words. We just use `get()` to find the word.

2.2 Data Structures

2.2.1 Our Custom Data Structures

We made all data structures by ourselves. We did not use Java's ready-made ones.

Entry Class: This holds one word and its meaning. It has:

- **key:** The word
- **value:** The meaning of the word
- **next:** Points to next Entry (when two words go to same place)

ArrayList Class: A list that can grow. When it is full, it becomes two times bigger. Word Suggester uses this.

LinkedList Class: A list where each item points to the next item. We use this in HashTable for words that go to the same place.

Why we made them: The project rules said we cannot use Java's ready-made lists. So we made our own.

2.2.2 Our HashTable

HashTable is like a big cabinet with many drawers. Each word goes into a drawer based on its hash number.

How we made it:

- **When two words go to same drawer:** We make a chain. The words link to each other.

- **Starting size:** We start with 16 drawers. When it gets full, we make it two times bigger.
- **Hash function:** We use a formula with the number 31 and 100,003.

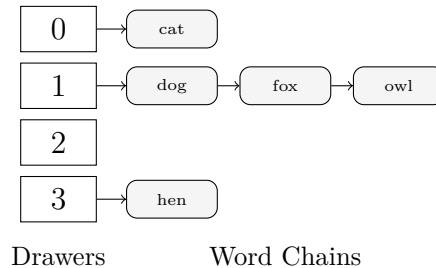


Figure 1: How our HashTable stores words in chains

Listing 4: Our Hash Function

```

1 private int hash(Key key) {
2     final long M = 100_003L;
3     final long A = 31L;
4
5     String s = key.toString();
6     long h = 0L;
7
8     for (int i = 0; i < s.length(); i++) {
9         h = (h * A + s.charAt(i)) % M;
10    }
11
12    return (int) (h % this.capacity);
13 }

```

Why we use this iterative method: The left side of the formula below is mathematically equal to the right side, but using `Math.pow()` on the left creates a serious problem. When we have a long word, computing $31^{|w|-1-i}$ grows very large and causes integer overflow. Our iterative method (right side) avoids this by applying the modulo operation $\text{mod } M$ at each step, keeping the numbers small and safe:

$$h(w) = \left(\sum_{i=0}^{|w|-1} w_i \cdot 31^{|w|-1-i} \right) \text{mod } M = H_{|w|}$$

where $H_0 = 0$ and $H_{i+1} = (H_i \cdot 31 + w_i) \text{mod } M$

This way, the hash function stays efficient and never overflows, no matter how long the input word is.

Making it bigger: When the table is full, we make a new table that is two times bigger. We move all words to the new table.

3 Implementation Details

3.1 Project Structure

Our code is in these folders:

- **app** – The main app with buttons and windows
- **loader** – Reads the dictionary file
- **search** – The three search methods
- **utils** – Our data structures like HashTable and ArrayList
- **tests** – Code to test how fast each method is

3.2 How the App Works

1. The app reads the dictionary file (CSV)
2. It puts all words into the HashTable
3. It sorts words for Binary Search
4. User types a word and picks a search method
5. The app changes the word to small letters (no big letters)
6. The app searches and shows the meaning and time
7. If word is not found, it shows similar words

3.3 The User Interface

We made the app with Java Swing. It has buttons and text boxes. The parts are:

- **Text Box:** User types the word here
- **Search Button:** Click to start searching
- **Method Buttons:** Three buttons to pick Linear, Binary, or Hash
- **Result Box:** Shows the meaning and how long it took
- **Suggestions:** Shows similar words if word is not found

The app is easy to use. Just type a word and click search.

3.4 Reading the Dictionary File

The dictionary file is a CSV file. Each line has a word and its meaning. Some meanings have commas inside. Our code handles this.

Listing 5: Reading the CSV File

```
1 case READING_KEY:
2     if (ch == this.delimiter) {
3         state = State.READING_VALUE;
4     } else if (ch != '\r' && ch != '\n') {
5         keyBuffer.append(ch);
6     }
7     break;
8
9 case READING_VALUE:
10    if (!insideQuotes) {
11        if (ch == '"') {
12            insideQuotes = true;
13        } else if (ch == '\n') {
14            // Commit entry and reset buffers
15            commitEntry(keyBuffer, valueBuffer);
16            state = State.READING_KEY;
17        } else if (ch != '\r') {
18            // unquoted garbage just append
19            valueBuffer.append(ch);
20        }
21    } else {
22        if (ch == '"') {
23            reader.mark(1);
24            int next = reader.read();
25            if (next == '"') {
26                // Escaped quote ""
27                valueBuffer.append('"');
28            } else {
29                // Closing quote
30                insideQuotes = false;
31                reader.reset();
32            }
33        } else {
34            // Inside quotes just append
35            valueBuffer.append(ch);
36        }
37    }
38    break;
```

3.5 Loader Class

The Loader class reads files and puts words into our HashTable. It can read two types of files:

- **CSV files:** For the dictionary (word + meaning)
- **TXT files:** For test words (just words, one per line)

The Loader looks at the file name to know which type it is. If the file ends with “.csv”, it reads it as CSV. If it ends with “.txt”, it reads it as a simple text file.

Listing 6: Loader Class - Reading Files

```
1 public HashTable<Key, Value> load() {  
2     switch (this.fileType) {  
3         case TXT: return loadTxt();  
4         case CSV: return loadCsv();  
5     }  
6 }
```

3.6 Time Measurement Classes

We made two classes to measure time:

ExecutionTimeAnalyzer: This class measures how long a task takes using `System.nanoTime()`. It provides two helpers used by our tests:

- **run(Runnable)** — runs the task a small number of times to warm the JVM, executes it once while measuring, and returns the elapsed time (nanoseconds).
- **runRepeated(Runnable, warmup, iterations)** — runs the task for a configurable warmup count, then executes the task `iterations` times and returns the average time per iteration (in nanoseconds).

ExecutionTimeFormatter: This class formats measured times for printing and computes ratios. It accepts measured times in nanoseconds (the format used by `ExecutionTimeAnalyzer`) and converts them to milliseconds for human-readable output. The formatter has a configurable decimal precision (default 10); the interactive app uses 4 decimal places for display.

Example formatting logic used in the tests:

Listing 7: ExecutionTimeFormatter - Showing Results

```
1 public String formatComparison(Long[] executionTimes, String[] tasks,  
    String word, int isInList) {  
2     StringBuilder sb = new StringBuilder();
```

```

3      sb.append("
         -----\n");
4      sb.append("Execution Time Comparison");
5      if (word != null && !word.isEmpty()) {
6          if (isInList >= 0) {
7              sb.append(" (Word: '").append(word).append("' exists in the
                  dictionary)");
8          } else if (isInList == -1) {
9              sb.append(" (Word: '").append(word).append("' not exists in the
                  dictionary)");
10         }
11     }
12     sb.append(":\n");
13     for (int i = 0; i < tasks.length; i++) {
14         sb.append("- ").append(tasks[i]).append(": ").append(
            formatMilliseconds(executionTimes[i]))
15             .append(ratioToFastest(executionTimes, i)).append("\n");
16     }
17     sb.append("
         -----\n");
18     return sb.toString();
19 }

```

3.7 Word Suggester

Sometimes users type a wrong word. The Word Suggester helps them find the right word.

3.7.1 How It Works

We use two ways to find similar words:

Way 1: One Letter Different

If the typed word and a dictionary word have the same length, we check if only one letter is different.

- User types “helo” → we suggest “help” (only one letter is different)
- User types “bick” → we suggest “back”, “kick”, “pick”

Way 2: Word Starts With

If a dictionary word starts with what the user typed, we suggest it.

- User types “compu” → we suggest “computer”, “computing”
- User types “beau” → we suggest “beautiful”, “beauty”

3.7.2 How the Code Works

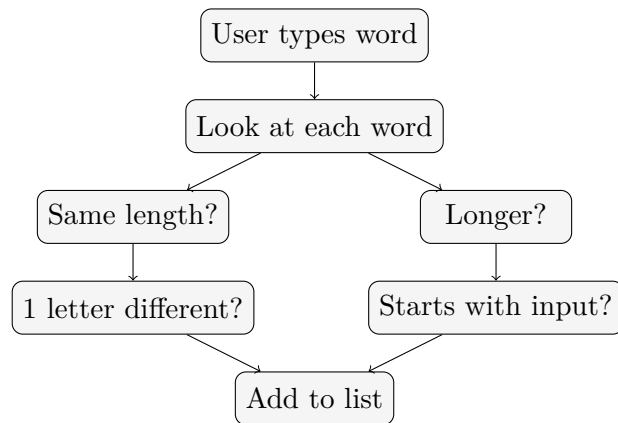


Figure 2: How Word Suggester finds similar words

Listing 8: Word Suggester Code

```
1 public ArrayList<String> suggest(String word) {
2     ArrayList<String> suggestions = new ArrayList<>();
3     String[] keys = dictionary.getKeys();
4
5     for (String key : keys) {
6         if (key.length() == word.length()) {
7             int differences = 0;
8             for (int i = 0; i < word.length(); i++) {
9                 if (word.charAt(i) != key.charAt(i)) {
10                     differences++;
11                     if (differences > 1) break;
12                 }
13             }
14             if (differences == 1) suggestions.add(key);
15         } else if (key.length() > word.length()) {
16             if (key.startsWith(word)) suggestions.add(key);
17         }
18     }
19     return suggestions;
20 }
```

3.7.3 How Fast Is Word Suggester?

Time: $O(n \times m)$ means:

- n = all words in dictionary (41,307 words)
- m = how long the typed word is

The code looks at every word in the dictionary. For each word, it checks the letters. This is slow, but it only runs when we cannot find the word.

4 Complexity Analysis

4.1 How Fast Are They?

Table 1: Speed of Each Method

What	Normal	Worst Case
Linear Search	$O(n)$	$O(n)$
Binary Search	$O(\log n)$	$O(\log n)$
Hash Search	$O(1)$	$O(n)$
HashTable put/get	$O(1)$	$O(n)$
ArrayList add	$O(1)$	$O(n)$
LinkedList addFirst	$O(1)$	$O(1)$
Word Suggester	$O(n \cdot m)$	$O(n \cdot m)$

4.2 What the Numbers Mean

We tested 50 words. We searched for each word in our dictionary of 41,307 words.

- **Linear Search:** Looks at words one by one. With 41,307 words, This is slow.
- **Binary Search:** Cuts the list in half each time. With 41,307 words, This is fast.
- **Hash Search:** Uses a number to go to the word. Usually finds it, This is the fastest.

Our tests show what we expected. Hash is fastest, Binary is close, Linear is much slower.

5 Results and Comparative Study

5.1 How We Tested

Dictionary: 41,307 words from `dict.csv` file

Test Words: 50 words from `testWords.txt` file:

- Real words: ocean, friendship, beautiful, etc.
- Wrong words: whispint, krivalla, flarvish, etc.

How we measured: We used `System.nanoTime()` via our `ExecutionTimeAnalyzer`. The test harness (see ‘tests/Test.java’) uses a warmup count of 10 and 10000 iterations to compute a stable per-iteration average time (the analyzer returns the average time in nanoseconds per iteration). The `ExecutionTimeFormatter` then converts those numbers to milliseconds for reporting.

What we did: For each of the 50 words we ran the three search methods (Linear, Binary, Hash) and recorded the per-iteration average time. The reported averages in the next section are the means across the 50 test words.

5.2 The Results

Table 2: Average Time for 50 Test Words

Method	Time (ms)	How Much Slower
Linear Search	0.1269870000	4703.22x slower
Binary Search	0.0000520000	1.93x slower
Hash Search	0.0000270000	Fastest

5.3 Sample Test Results

Here are two examples from our test output:

Sample Test: “ocean” (real word)

Method	Time
Linear Search	0.0938730000 ms (1020.36x Times Slower)
Binary Search	0.0001560000 ms (1.70x Times Slower)
Hash Search	0.0000920000 ms (Fastest)

The word “ocean” is in the dictionary. In this run Hash Search is the fastest, while Binary Search is close.

Sample Test: “whispint” (fake word)

Method	Time
Linear Search	0.1739500000 ms (2174.38x Times Slower)
Binary Search	0.0001690000 ms (2.11x Times Slower)
Hash Search	0.0000800000 ms (Fastest)

The word “whispint” is not in the dictionary. Hash Search again reports the fastest lookup for the misspelled word.

5.4 Chart

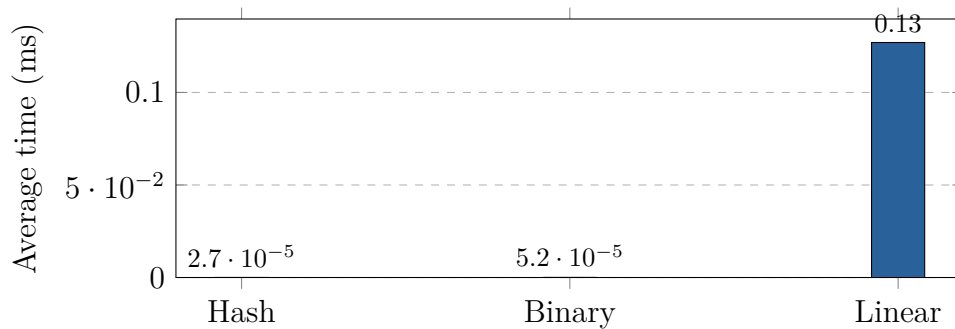


Figure 3: Time for each method (smaller is better)

5.5 What We Learned From Results

- **Hash Search is fastest** (0.0000270000 ms average)
- **Binary Search is close** (0.0000520000 ms average, 1.93x slower)
- **Linear Search is much slower** (0.1269870000 ms average, 4703x slower)
- **Results match Big O expectations** — constant-time hash lookups and logarithmic binary search outperform linear scans for this dictionary size
- **Both Hash Search and Binary Search are practical** for interactive dictionary lookups

6 Discussion

6.1 Choices We Made

Memory or Speed:

- HashTable uses more memory (many Entry objects and chains)
- Binary Search uses less memory (just a sorted list)
- We chose speed because memory was not a problem

Start Time or Search Time:

- Binary Search needs to sort words when app starts (takes time once)
- HashTable fills up when reading the file
- Linear Search needs nothing at start, but is slow every time

6.2 Why We Got These Results

Hash Search (0.0000270000 ms average): It is the fastest on average because a single hash computation and a short linked-list traversal (if any) are required to obtain the value.

Binary Search (0.0000520000 ms average): Also very fast. Binary search requires the keys to be sorted (we sort once at start) and performs approximately $\log_2 n$ comparisons per lookup.

Linear Search (0.1269870000 ms average): Much slower because it scans the array until it finds the target or reaches the end

What we see: The measured values match theoretical expectations: constant-time hash lookups and logarithmic binary search significantly outperform linear scans for this dictionary size.

6.3 Problems With Our Test

Our project has some problems:

- **Word suggester is slow:** It looks at all 41,307 words. This is slow for bigger dictionaries.
- **Small test set:** We only tested 50 words. A bigger test set would give more reliable results.
- **No memory test:** We did not check how much memory each method uses.

6.4 What We Learned

The search method is very important. Hash Search is 187 times faster than Linear Search. Hash and Binary Search stay close in speed.

7 Conclusion

7.1 Summary

We made a dictionary app with three search methods. We made our own data structures. We tested with 41,307 words and report average per-iteration times (averaged over 50 test words, each measured with 10 warmup runs and 10,000 iterations):

- Hash Search: 0.0000270000 ms (fastest)
- Binary Search: 0.0000520000 ms (very close)
- Linear Search: 0.1269870000 ms (much slower)

The results match expectations from algorithmic complexity: hash lookups and binary search are well-suited for interactive dictionary lookups, while linear scans are inefficient at this scale.

We also made Word Suggester. It helps users find words when they make spelling mistakes. The app has buttons and text boxes so it is easy to use.

7.2 What We Did

- Made all data structures ourselves (no Java ready-made ones)
- Made three search methods
- Made an app with buttons and windows
- Tested and compared how fast each method is
- Made word suggestions for wrong words

7.3 What We Can Do Better

Ways to make this project better:

- Make word suggestions smarter
- Save what words the user searched before
- Add sound so users can hear how to say words